

CVM++

How cvmpp runs your program — every stage explained for a reader with no compiler background

A self-contained guide — read top to bottom.

github.com/Dragon9-9/cvmpp · Satish Kumar Munda · MIT License

What this guide explains

You already know **how to run** CVM++ from the Project and Build Guide. This guide answers **what happens inside the computer** when you run:

```
./build/cvmpp examples/hello.cvm
```

We follow one line of data from text on disk to numbers on the screen. Each section names the input, the output, the files involved, and common mistakes at that stage.

No table of contents — use section headings if you jump around.

The big picture (read this first)

When cvmpp runs, it does not “interpret” your file by re-reading English-like text over and over. It converts the file once into bytecode, then runs that bytecode like a tiny CPU.

In plain words: Think of bytecode as a checklist of simple steps only the VM understands. Your .cvm file is the human-friendly version; bytecode is the machine-friendly version.

Who calls whom

```
main.cpp
  calls compile_frontend(source)    in compile.cpp
    calls Lexer::tokenize()
    calls Parser::parse()
    calls Compiler::compile()
  then calls execute(chunk)         in vm.cpp
    calls VirtualMachine::run()
```

If any compile step fails, the VM never runs. If the VM fails, you get exit code 2 but compilation already succeeded.

One table for all stages

Stage	Input	Output	Main file
Lexer	Source string	List of tokens	lexer.cpp
Parser	Tokens	AST Program	parser.cpp
Compiler	AST	Bytecode bytes	compiler.cpp
VM	Bytecode	Printed lines	vm.cpp

Stage 1 — Lexer (characters to tokens)

What is a token?

A token is a labeled chunk of the source. For input `let x = 42;` the lexer produces something like:

Token type	Text (lexeme)
Let	let
Identifier	x
Assign	=
Integer	42

Semicolon	;
-----------	---

The parser only sees this list — not the original string spacing.

What the lexer does (algorithm in plain steps)

1. Skip spaces, tabs, newlines (track line number for errors).
2. If you see `//`, skip until end of line (comment — ignored).
3. If digit → read integer; check overflow; emit Integer or error.
4. If letter → read identifier; if it matches a keyword (`let`, `fn`, ...), emit keyword token; else Identifier.
5. If `=` → `==` is two equals; single `=` is Assign.
6. Similar rules for `!=`, `<=`, `>=`, `<`, `>`, `+`, `-`, `*`, `/`, `(`, `)`, `{`, `}`, `;`, `,`
7. At end of file → emit Eof.

When the lexer reports an error

Examples: unknown character `@`, number too big for 64-bit, null byte in file. Errors say LEXER phase and show line/column with a caret under the problem.

Stage 2 — Parser (tokens to tree)

What is an AST?

AST means Abstract Syntax Tree. “Abstract” because punctuation like semicolons is dropped; “tree” because nodes point to children.

Example structure for `print 1 + 2;;`

```
PrintStmt
├─ BinaryExpr (+)
│   ├── IntLiteral(1)
│   └─ IntLiteral(2)
```

The compiler will later walk this tree and know: first evaluate 1, then 2, then add, then print.

Program shape

Every file becomes a Program with two lists:

1. functions — all `fn name(...) { ... }` declarations (parsed first).
2. statements — the “main script” that runs after functions are defined.

Expression precedence (which operator binds tighter)

When you write `1 + 2 * 3`, multiplication happens first because `*` has higher precedence than `+`.

The parser encodes this by calling smaller functions first:

Level	Operators
Lowest	<code>==</code> <code>!=</code>
Next	<code><</code> <code>></code> <code><=</code> <code>>=</code>
Next	<code>+</code> <code>-</code>
Highest	<code>*</code> <code>/</code>
Unary	<code>-</code> on one number

Assignment ($x = 5$) is a statement, not an expression in the middle of another expression.

Parser errors

Wrong syntax: $10 = x$ (left side of $=$ must be a variable name), missing $\}$, extra tokens. Phase **PARSER**. After an error, the parser may synchronize to the next $;$ to avoid hundreds of follow-up messages.

Stage 3 — Compiler (tree to bytes)

What is bytecode?

A sequence of bytes in `BytecodeChunk::code`. Each byte is often an opcode (command). Some opcodes are followed by extra bytes: a number, a variable index, or a jump address.

Example **idea** for `print 1 + 2;` (offsets simplified):

Bytes mean	Effect
PUSH_INT 1	Put 1 on stack
PUSH_INT 2	Put 2 on stack
ADD	Pop two, push sum 3
PRINT	Pop and show 3
HALT	Stop

Memory layout (must understand this)

The compiler emits bytes in this exact order:

1. Byte 0: JUMP to skip ahead (operand filled in later).
2. Next: bodies of all fn functions (may use `LOAD_LOCAL`, `RETURN`, ...).
3. Main entry label: top-level statements from your file.
4. End: `HALT`.

Why a jump at the start?

Function code must live somewhere in the byte array. It is placed first. Without the jump, the VM would start executing inside a function without a `CALL` — and the first `LOAD_LOCAL` would crash. The jump lands on your real “main” script.

How **if** becomes jumps

1. Compile condition $\rightarrow$ boolean on stack.
2. `JUMP_IF_FALSE` to else branch (address patched later).
3. Compile “then” part.
4. Optional `JUMP` over else; compile else; patch addresses.

Same idea as assembly language you might see in a computer architecture course — but generated automatically from `if`.

How **while** becomes jumps

1. Mark loop start address.
2. Compile condition; `JUMP_IF_FALSE` to exit.
3. Compile body.
4. `JUMP` back to loop start; patch exit address.

Functions in bytecode

For each fn:

1. Record name, entry address (current byte offset), arity (parameter count).
2. Map parameter names to local slots 0, 1, ...
3. Compile function body.

For each call `f(1, 2)`:

1. Compile arguments left-to-right (they end up on stack in order).
2. Emit CALL with function index and argument count 2.

Compiler errors

Call undefined function, wrong number of arguments, return without value. Phase COMPILER.

Stage 4 — Virtual machine (bytes to behavior)

What the VM keeps in memory

Piece	Purpose
ip_	Which bytecode byte to read next — like a program counter.
stack_	Temporary values while evaluating expressions.
globals_	Variable values in file mode (by index).
frames_	Stack of active function calls.
session_	Optional: REPL map from name to value.

Runtime types: only integer (`int64_t`) and boolean. `true + 5` is rejected with a clear type error.

The run loop (simple pseudocode)

repeat:

```
  if too many steps → error (infinite loop protection)
  read one opcode byte
  do what that opcode means (push, pop, jump, call, ...)
```

until HALT or error

Maximum stack depth: 65536. Maximum steps: 1,000,000 per run.

Globals: file vs REPL

1. File mode: `LOAD_VAR` / `STORE_VAR` use an array slot per global name index. Unread globals error until assigned.
2. REPL mode: same instructions, but values live in `VmSession::variables` keyed by name, so `let x = 1` persists on the next line you type.

Opcode reference (what each instruction does)

For binary ops, the top of the stack is the right operand.

Opcode	Meaning
PUSH_INT	Read 8-byte integer from bytecode; push on stack.
PUSH_BOOL	Read 0/1 byte; push true/false.

LOAD_VAR	Push global value by name index.
STORE_VAR	Pop; store into global.
LOAD_LOCAL	Push local slot in current function frame.
STORE_LOCAL	Pop; store into local slot.
ADD SUB MUL DIV	Pop b, pop a; push result; DIV checks zero.
EQ NE LT GT LE GE	Comparisons; push boolean.
INPUT	Read line from stdin; push integer.
PRINT	Pop; append text to program output.
JUMP	Set IP to 4-byte address.
JUMP_IF_FALSE	Pop bool; if false, jump.
CALL	Pop argc values; start function at its address.
RETURN	Pop return value; end frame; push value to caller.
HALT	Stop running.

How function calls work (step by step)

Suppose main calls `factorial(5)`:

1. Main code pushes 5 on the stack.
2. CALL pops 5 into `frame.locals[0]`, saves return address, sets IP to `factorial`'s entry.
3. Inside `factorial`, `LOAD_LOCAL 0` reads `n`.
4. RETURN pops the return value, restores IP to after the CALL, pushes result on stack for main.
5. Main continues (e.g. PRINT).

Recursive calls push another frame on `frames_;` each RETURN goes back one level.

Worked example: `print 1 + 2;`

| Step | What VM does | Stack after | |---|-----|-----| | 1 | Push 1 | [1] | | 2 | Push 2 | [1, 2] | | 3 | ADD → 3 | [3] | | 4 | PRINT → output "3" | [] |

Worked example: `factorial(5) → 120`

```
fn factorial(n) {
  if (n <= 1) { return 1; }
  return n * factorial(n - 1);
}
print factorial(5);
```

1. **Compile time:** parser builds two functions list entry + main statements. Compiler records `factorial`'s entry address, emits jump-at-start, emits `factorial` body, emits main that pushes 5 and CALLs `factorial`, then PRINT.
2. **Run time:** CALL creates frame with `n=5`. Recursive calls until `n` is 1. Each RETURN brings a product back. Outer RETURN leaves 120 on stack. PRINT shows 120.

Trace table for `print 1 + 2;` (repeat for clarity)

Step	Instruction	Stack
------	-------------	-------

1	PUSH_INT 1	[1]
2	PUSH_INT 2	[1,2]
3	ADD	[3]
4	PRINT	output 3

The compile_frontend function (glue code)

This function in compile.cpp is the spine of the toolchain:

```
FrontEndResult compile_frontend(std::string source) {
    // 1. Lexer – if fail, return immediately
    // 2. Parser – if fail, return immediately
    // 3. Compiler – produces BytecodeChunk
    // VM is NOT called here – main.cpp calls execute() after
}
```

FrontEndResult holds source text (for error display), tokens (for debug), AST, bytecode, and later VM output. Method ok() is false if any stage recorded an error.

Opcode bytes (hex) for readers who want detail

Opcode	Hex	Extra bytes after opcode
PushInt	0x01	8-byte signed integer
PushBool	0x02	1 byte 0 or 1
LoadVar	0x10	2-byte name index
StoreVar	0x11	2-byte name index
LoadLocal	0x12	1-byte slot
StoreLocal	0x13	1-byte slot
Add-Div	0x20–0x23	none
Jump	0x40	4-byte offset
JumpIfFalse	0x41	4-byte offset
Call	0x42	2-byte fn index + 1-byte argc
Return	0x43	none
Halt	0xFF	none

How errors look

The UI prints phase name, line/column, message, hint, and a source line with ^ under the column.

Phases: LEXER, PARSER, COMPILER, VM, REPL (unknown command).

The C++ program should not crash on your mistakes — it exits with code 1 or 2.

Example error message (what you might see)

```
[X] PARSER error at 2:1: unexpected token ...
    hint: expected ...
```

```
| 10 = x;  
|   ^  
|
```

The phase tells you **which part** to study: lexer (characters), parser (grammar), compiler (internal), VM (while running).

File mode vs REPL mode (architecture difference)

Question	File `cvmpp f.cvm`	REPL `cvmpp`
Where globals live	Array indexed by name pool	Map string → value in VmSession
Between runs	All globals forgotten	let on line 1 visible on line 2
Best for	Scripts, CI, homework files	Experiments, calculator style

Same bytecode instructions (LOAD_VAR) — different storage behind them depending on whether session pointer is null.

How to debug (see inside the machine)

```
./build/cvmpp -d examples/hello.cvm
```

Study output in this order:

1. Token table — did the lexer split words correctly?
2. AST — does the tree match what you meant?
3. Bytecode — are jumps and calls reasonable?
4. Runtime output — final numbers and any VM errors.

In REPL: `:disasm file.cvm` compiles without running (good for seeing bytecode only).

How modules depend on each other

main.cpp uses compile_frontend which uses lexer, parser, compiler. Compiler and VM both use BytecodeChunk. Parser uses ast.hpp. Everything links into one cvmpp binary — no plugins.